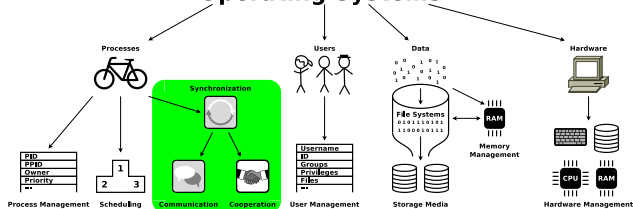


- At the end of this slide set, you know/understand...
 - what **critical sections** and **race conditions** are
 - what **synchronization** is
 - how **signaling** influences the execution order of the processes
 - how critical sections can be secured via **blocking**
 - what problems (**starvation** and **deadlocks**) may arise from blocking
 - how **deadlock detection with matrices** works
 - different options to implement **communication** between processes:
 - **Shared memory, Message queues, Pipes, Sockets**
 - different options to implement **cooperation** between processes
 - how critical sections can be protected via **semaphores** (and **mutex**)

Operating Systems



Interprocess Communication (IPC)

- Processes do not only carry out read and write operations on data, but also:
 - call each other
 - wait for each other
 - coordinate with each other
 - In short: They must **interact** with each other
- Important questions regarding **interprocess communication** (IPC):
 - How can a process transmit information to others?
 - How can multiple processes access shared resources?

Question: What is the situation here with threads?

- For threads, the same challenges and solutions exist as for interprocess communication with processes
- Only the communication between the threads of a process is no problem because they operate in the same address space

7. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

Therac-25: Race Condition with tragic Result (1/2)

- Therac-25 is a linear particle accelerator for the radiation therapy of cancer tumors
- Mid-1980s: In the United States some accidents happened because of poor programming and quality assurance
 - Some patients got an up to 100 times increased radiation dose

An Investigation of the Therac-25 Accidents. Nancy Leveson, Clark S. Turner. IEEE Computer, Vol. 26, No. 7, July 1993, S.18-41
http://courses.cs.vt.edu/~cs3604/lib/Therac_25/Therac_1.html

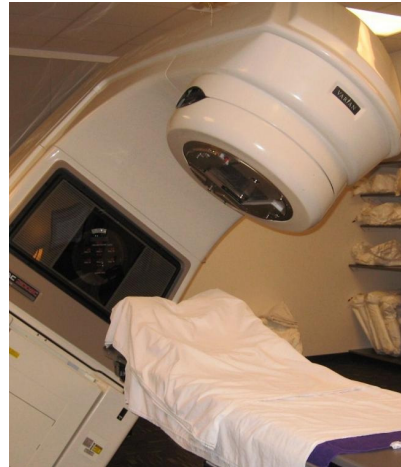


Image source: Google image search.
Frequently shown picture in this context.
(author and license: unknown)

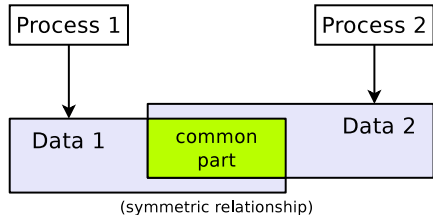
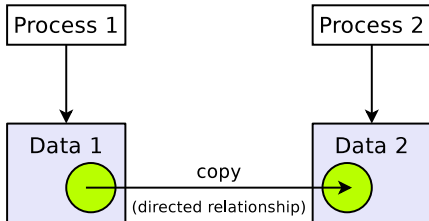
- A race condition (“Texas-Bug”) led to incorrect settings of the device and consequently to increased radiation doses.
 - The control process did not synchronize correctly with the user interface process
 - The error occurred only during a quick input correction (time window: 8 seconds) by the user
 - During testing the error did not occur because experience (routine) was required to operate the device this fast

"Once the data entry phase was marked complete, the magnet setting phase began. However, if a specific sequence of edits was applied in the Data Entry phase during the 8 second magnet setting phase, the setting was not applied to the machine hardware, due to the value of the completion variable. The UI would then display the wrong mode to the user, who would confirm the potentially lethal treatment."

https://www.dssz.informatik.tu-cottbus.de/information/slides_studis/ss2009/mehner_RisikoComputer_zs09.pdf
 Killer Bug. Therac-25: Quick-and-Dirty: https://www.viva64.com/en/b/0438/
 Killed by a machine: The Therac-25: https://hackaday.com/2015/10/26/killed-by-a-machine-the-therac-25/

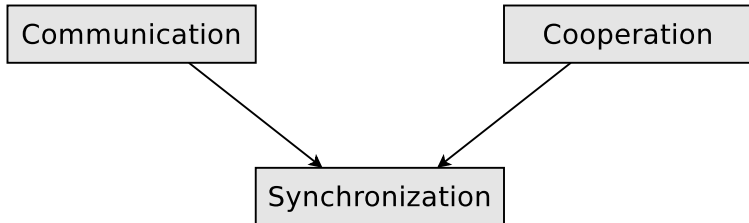
1. *Journal of Management Studies*, 1996, 33, 1, 1-14.

1000



Forms of Interaction

- Communication and cooperation are based on synchronization
 - Synchronization is the most elementary form of interaction
 - Reason: communication and cooperation need a synchronization between the interacting partners to obtain correct results
 - Therefore, we first discuss the **synchronization**



The diagram illustrates the implementation of a semaphore using two binary locks and a shared variable s .

Process 1 and Process 2: Each process has a vertical timeline. Process 1's timeline shows a light blue bar labeled X between its `lock(s)` and `unlock(s)` operations. Process 2's timeline shows a light blue bar labeled Y between its `lock(s)` and `unlock(s)` operations. Arrows indicate that both processes can acquire either lock, but only one can be in the critical section at a time.

Semaphore Operations:

- unlock(s):** This operation resets the shared variable s to 0. It then enters a loop: "process waiting?". If the answer is "yes", it proceeds to "deblock process" and continues. If the answer is "no", it loops back to "process waiting?".
- lock(s):** This operation enters a loop: "is s set?". If the answer is "yes", it proceeds to "block process" and loops back to "is s set?". If the answer is "no", it proceeds to "set s" and continues.

- Alternative 1: Implementation of locking with the signals SIGSTOP (No. 19) and SIGCONT (No. 18)
 - With SIGSTOP another process can be stopped
 - With SIGCONT another process can be reactivated

Conditions for Deadlock Occurrence

System Deadlocks. E. G. Coffman, M. J. Elphick, A. Shoshani. *Computing Surveys*, Vol. 3, No. 2, June 1971, P.67-78
http://people.cs.umass.edu/~mcorner/courses/691J/papers/TS/coffman_deadlocks/coffman_deadlocks.pdf

- A deadlock situation can arise if these conditions are all fulfilled
 - **Mutual exclusion**
 - At least 1 resource is occupied by exactly 1 process or is available
 $\implies$ non-sharable
 - **Hold and wait**
 - A process, which currently occupies at least 1 resource, requests additional resources which are being held by another process
 - **No preemption**
 - Resources, which are occupied by a process cannot be deallocated by the operating system, but only released by the holding process voluntarily
 - **Circular wait**
 - A cyclic chain of processes exists
 - Each process requests a resource that the next process in the chain occupies.
- If one of these conditions is not fulfilled, no deadlock can occur

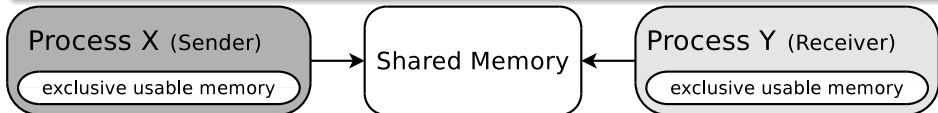
Deadlock Detection with Matrices

- One drawback of deadlock detection with resource graphs is that only individual resources can be represented with it
 - If multiple copies (instances) of a resource exist, then graphs are not suited for the visualization and detection of deadlocks
 - If multiple copies of a resource exist, a matrix-based algorithm can be used, which requires 2 vectors and 2 matrices
- We specify 2 vectors
 - **Existing resource vector**
 - Indicates the number of existing resources of each class
 - **Available resource vector**
 - Indicates the number of free resources of each class
- Additionally 2 matrices are required
 - **Current allocation matrix**
 - Indicates, which resources are currently occupied by the processes
 - **Request matrix**
 - Indicates, which resources the processes would like to occupy

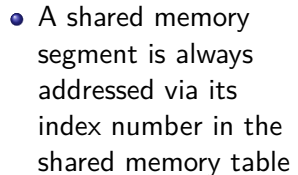
Shared Memory

- Interprocess communication via a shared memory is also called **memory-based communication**
- **Shared memory segments** are memory areas, which can be accessed by multiple processes
 - These memory areas are located in the address space of multiple processes
- The processes need to coordinate the access operations by themselves and ensure that their memory requests are mutually exclusive
 - A receiver process cannot read data from the shared memory, before the sender process has finished its current write operation
 - If access operations are not coordinated carefully $\implies$ inconsistencies

In all other forms of interprocess communication, the operating system takes care of the synchronization of the access operations



- Linux/UNIX operating systems contain a **shared memory table**, which contains information about the existing shared memory segments
 - This information includes: Start address in memory, size, owner (username and group) and privileges



- Advantage: A shared memory segment which is not attached to a process, is not erased by the operating system automatically

26/76

Working with Shared Memory (System V vs. POSIX)

Linux/UNIX operating systems provide 4 system calls for working with shared memory

- `shmget()`: Create a shared memory segment or access an existing one
- `shmat()`: Attach a shared memory segment to a process
- `shmdt()`: Detach a shared memory segment from a process
- `shmctl()`: Request status information (e.g. privileges) of a shared memory segment, modify or erase it
- The command `ipcs` provides information about existing shared memory segments (System V)

One example of working with shared memory segments in Linux can be found on the website of this course

- Some developers prefer the System V API and others the POSIX API...

C function calls for working with POSIX shared memory segments (some defined in the header file `mman.h`)

- `shm_open()`: Create a shared memory segment or access an existing one
- `ftruncate()`: Specify the size of a shared memory segment
- `mmap()`: Attach a shared memory segment to a process
- `munmap()`: Detach a shared memory segment from a process
- `close()`: Close the descriptor of a shared memory segment
- `shm_unlink()`: Erase a segment
- In Linux, POSIX shared memory segments can be found in the `/dev/shm` directory

One example of working with POSIX shared memory segments in Linux can be found on the website of this course


```

1 #include <sys/types.h>
2 #include <sys/ipc.h>
3 #include <sys/shm.h>
4 #include <stdio.h>
5 #define MAXMEMSIZE 20
6
7 int main(int argc, char **argv) {
8     int shared_memory_id = 12345;
9     int returncode_shmget;
10    char *sharedmempointer;
11
12    // Create shared memory segment or access an existing one
13    returncode_shmget = shmget(shared_memory_id, MAXMEMSIZE, IPC_CREAT | 0600);
14    ...
15
16    // Attach shared memory segment
17    sharedmempointer = shmat(returncode_shmget, 0, 0);
18    if (sharedmempointer==(char *)-1) {
19        printf("Unable to attach the shared memory segment.\n");
20        perror("shmat");
21    } else {
22        printf("The shared memory segment has been attached %p\n", sharedmempointer);
23    }
24 }
25 }

```

```
----- Shared Memory Segments -----
key          shmid      owner      perms      bytes      nattch     status
0x00003039  56393780    bnc        600        20         1
```


Detach a (System V) Shared Memory Segment (in C)

```
1 #include <sys/types.h>
2 #include <sys/ipc.h>
3 #include <sys/shm.h>
4 #include <stdio.h>
5 #define MAXMEMSIZE 20
6
7 int main(int argc, char **argv) {
8     int shared_memory_id = 12345;
9     int returncode_shmget;
10    int returncode_shmdt;
11    char *sharedmempointer;
12
13    // Create shared memory segment or access an existing one
14    returncode_shmget = shmget(shared_memory_id, MAXMEMSIZE, IPC_CREAT | 0600);
15    ...
16
17    // Attach the shared memory segment
18    sharedmempointer = shmat(returncode_shmget, 0, 0);
19    ...
20
21    // Detach the shared memory segment
22    returncode_shmdt = shmdt(sharedmempointer);
23    if (returncode_shmdt < 0) {
24        printf("Unable to detach the shared memory segment.\n");
25        perror("shmdt");
26    } else {
27        printf("The shared memory segment has been detached.\n");
28    }
29 }
30 }
```

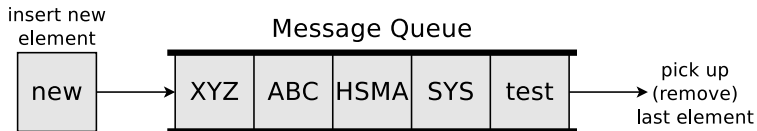
Erase a (System V) Shared Memory Segment (in C)

```

1 #include <sys/types.h>
2 #include <sys/ipc.h>
3 #include <sys/shm.h>
4 #include <stdio.h>
5 #define MAXMEMSIZE 20
6
7 int main(int argc, char **argv) {
8     int shared_memory_id = 12345;
9     int returncode_shmget;
10    int returncode_shmctl;
11    char *sharedmempointer;
12
13    // Create shared memory segment or access an existing one
14    returncode_shmget = shmget(shared_memory_id, MAXMEMSIZE, IPC_CREAT | 0600);
15    ...
16
17    // Erase shared memory segment
18    returncode_shmctl = shmctl(returncode_shmget, IPC_RMID, 0);
19    if (returncode_shmctl == -1) {
20        printf("Unable to erase the shared memory segment.\n");
21        perror("semctl");
22    } else {
23        printf("The shared memory segment has been erased.\n");
24    }
25 }
26 }

```

- Are linked lists with messages
- Operate according to the FIFO principle
 - However, the retrieval order can be controlled
 - System V message queues allow messages to be selected by type
 - POSIX message queues take message priorities into account
- Processes can store messages inside and fetch them up from there
- Benefit: Even after the termination of the process, which created the message queue, the data inside the message queue stays available



Linux/UNIX operating systems provide 4 system calls for working with message queues (System V)

- `msgget()`: Create a message queue or access an existing one
- `msgsnd()`: Write message into message queues ($\implies$ send operation)
- `msgrcv()`: Read message from message queues ($\implies$ receive operation)
- `msgctl()`: Request status information (e.g. privileges) of a message queue, modify or erase it
- The command `ipcs` provides information about existing SystemV message queues

Create (System V) Message Queues (in C)

```

1 #include <stdlib.h>
2 #include <sys/types.h>
3 #include <sys/ipc.h>
4 #include <stdio.h>
5 #include <sys/msg.h>
6
7 int main(int argc, char **argv) {
8     int returncode_msgget;
9
10    // Create message queue or access an existing one
11    // IPC_CREAT => create a message queue, if it does not still exist
12    // 0600 = Access privileges for the new message queue
13    returncode_msgget = msgget(12345, IPC_CREAT | 0600);
14    if(returncode_msgget < 0) {
15        printf("Unable to create the message queue.\n");
16        exit(1);
17    } else {
18        printf("The message queue 12345 with the ID %i has been created.\n",
19               returncode_msgget);
20    }
21 }

```

```
$ ipcs -q
----- Message Queues -----
key          msqid          owner          perms          used-bytes      messages
0x00003039  98304          bnc           600           0              0

$ printf "%d\n" 0x00003039          # Convert from hexadecimal to decimal
12345
```

Write Messages into (System V) Message Queues (in C)

```

1 #include <stdlib.h>
2 #include <sys/types.h>
3 #include <sys/ipc.h>
4 #include <stdio.h>
5 #include <sys/msg.h>
6 #include <string.h>                                // This header file is required for strcpy()
7
8 struct msgbuf {                                     // Template of a buffer for msgsnd and msgrcv
9     long mtype;                                     // Message type
10    char mtext[80];                                  // Send buffer
11 } msg;
12
13 int main(int argc, char **argv) {
14     int returncode_msgget;
15
16     // Create message queue or access an existing one
17     returncode_msgget = msgget(12345, IPC_CREAT | 0600);
18     ...
19
20     msg.mtype = 1;                                   // Specify the message type
21     strcpy(msg.mtext, "Testnachricht");              // Write the message into the send buffer
22
23     // Write a message into the message queue
24     if (msgsnd(returncode_msgget, &msg, strlen(msg.mtext), 0) == -1) {
25         printf("Unable to write the message into the message queue.\n");
26         exit(1);
27     }
28 }

```

- The message type (a positive integer value) is specified by the user

Result of writing a Message into a Message Queue

- Before...

```
$ ipcs -q
----- Message Queues -----
key          msqid      owner      perms      used-bytes   messages
0x00003039  98304      bnc        600         0             0
```

- Afterwards...

```
$ ipcs -q
----- Message Queues -----
key          msqid      owner      perms      used-bytes   messages
0x00003039  98304      bnc        600         80            1
```

Pick a Message from a (System V) Message Queue (in C)

```
1 #include <stdlib.h>
2 #include <sys/types.h>
3 #include <sys/ipc.h>
4 #include <stdio.h>
5 #include <sys/msg.h>
6 #include <string.h>           // This header file is required for strcpy()
7 struct msgbuf {              // Template of a buffer for msgsnd and msgrcv
8     long mtype;               // Message type
9     char mtext[80];           // Send buffer
10 } msg;
11
12 int main(int argc, char **argv) {
13     int returncode_msgget, returncode_msgrcv;
14     msg receivebuffer;         // Create a receive buffer
15
16     // Create message queue or access an existing one
17     returncode_msgget = msgget(12345, IPC_CREAT | 0600)
18
19     msg.mtype = 1;             // Pick the first message of type 1
20     // MSG_NOERROR => The message will be truncated when it is too long
21     // IPC_NOWAIT  => Do not block the process if no message exists
22     returncode_msgrcv = msgrcv(returncode_msgget, &msg, sizeof(msg.mtext),
23                                msg.mtype, MSG_NOERROR | IPC_NOWAIT);
24     if (returncode_msgrcv < 0) {
25         printf("Unable to pick a message from the message queue.\n");
26         perror("msgrcv");
27     } else {
28         printf("This message was picked from the message queue: %s\n", msg.mtext);
29         printf("The received message is %i characters long.\n", returncode_msgrcv);
30     }
31 }
```

Erase a (System V) Message Queue (in C)

```
1 #include <stdlib.h>
2 #include <sys/types.h>
3 #include <sys/ipc.h>
4 #include <stdio.h>
5 #include <sys/msg.h>
6
7 int main(int argc, char **argv) {
8     int returncode_msgget;
9     int returncode_msgctl;
10
11     // Create message queue or access an existing one
12     returncode_msgget = msgget(12345, IPC_CREAT | 0600);
13     ...
14
15     // Erase message queue
16     returncode_msgctl = msgctl(returncode_msgget, IPC_RMID, 0);
17     if (returncode_msgctl < 0) {
18         printf("Unable to erase the message queue with the ID %i.\n", returncode_msgget);
19         perror("msgctl");
20         exit(1);
21     } else {
22         printf("The message queue with the ID %i has been erased.\n", returncode_msgget);
23     }
24     exit(0);
25 }
```

One example of working with System V message queues in Linux can be found on the website of this course

Message Queues in Linux (System V vs. POSIX)

- The functions described so far for working with message queues are part of the **System V** interface
- Some developers prefer the System V API and others the POSIX API... _(ツ)_/

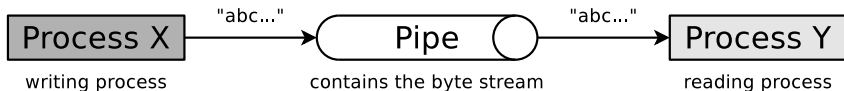
C function calls for POSIX message queue specified in the header file `mqueue.h`

- `mq_open()`: Create a message queue or access an existing one
- `mq_send()`: Write (send) a message into a message queue. Blocking operation
- `mq_timedsend()`: Write (send) a message into a message queue. Blocking operation with a timeout
- `mq_receive()`: Read (receive) a message from a message queue. Blocking operation
- `mq_timedreceive()`: Read (receive) a message from a message queue. Blocking operation with a timeout
- `mq_getattr()`: Request the attributes of a message queue. These are: number of messages in the queue, maximum message size, maximum number of messages...
- `mq_setattr()`: Modify the attributes of a message queue
- `mq_notify()`: Notify the process as soon as a message is available
- `mq_close()`: Close a message queue
- `mq_unlink()`: Erase a message queue
- POSIX message queues are created in Linux in the folder `/dev/mqueue`

One example of working with POSIX message queues in Linux can be found on the website of this course

Anonymous Pipes (1/2)

- Pipes can be **anonymous pipes** or **named pipes** (see slide 44)
- An **anonymous pipe**. . .
 - is a buffered unidirectional communication channel between 2 processes
 - If communication in both directions shall be possible at the same time, 2 pipes are necessary – one for each communication direction
 - operates according to the FIFO principle
 - has a limited capacity
 - Pipe = filled $\implies$ the writing process gets blocked
 - Pipe = empty $\implies$ the reading process gets blocked
 - is created with the system call `pipe()`
 - During this process, the kernel of the operating system creates an Inode ($\implies$ slide set 6) and 2 file descriptors (*handles*)
 - Processes access the access identifiers with `read()` and `write()` system calls (or standard library functions) for reading data from or writing data into the pipe



Anonymous Pipes (2/2)

- When child processes are created with `fork()`, the child processes also inherit access to the file descriptors
- **Anonymous pipes** allow process communication only between closely related processes
 - Only processes, which are closely related via `fork()` can communicate with each other via anonymous pipes
 - If the last process, which has access to an anonymous pipe, terminates, the pipe gets erased by the operating system

Overview of the pipes in Linux/UNIX: `ls -l | grep pipe`

Default size and maximum size of anonymous pipes in Linux

- Default size $\implies$ `int default_size = fcntl(pipefd[1], F_GETPIPE_SZ);`
Typically in Linux: 65536 bytes
- Modify the default size $\implies$ `fcntl(pipefd[1], F_SETPIPE_SZ, <size_in_bytes>);`
- Maximum allowed size (in bytes) is specified in the file `/proc/sys/fs/pipe-max-size`
Typically in Linux: 1048576 = 1 MB

Source: <https://manpages.ubuntu.com/manpages/jammy/en/man7/pipe.7.html>

Anonymous Pipe Example (in C) – Part 1/2

You can monitor the anonymous pipe in Linux/UNIX via `lsuf -n -P | grep <PID>` and inside the directory `/proc/<PID>/fd`

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4
5 void main() {
6     int pid_of_child;
7     // Create handles for the pipe to read (testpipe[0]) and write (testpipe[1])
8     int testpipe[2];
9
10    // Create anonymous pipe testpipe
11    if (pipe(testpipe) < 0) {
12        printf("Unable to create the anonymous pipe.\n");
13        // Terminate process
14        exit(1);
15    } else {
16        printf("Created the anonymous pipe testpipe.\n");
17    }
18
19    // Create a child process
20    pid_of_child = fork();
21
22    if (pid_of_child < 0) {
23        perror("Unable to create the child process!\n");
24        // Terminate process
25        exit(1);
26    }
```

Anonymous Pipe Example (in C) – Part 2/2

```
27 // Parent process
28 if (pid_of_child > 0) {
29     printf("Parent process: PID: %i\n", getpid());
30     // Block the read channel of the anonymous pipe testpipe
31     close(testpipe[0]);
32     char message[] = "Testnachricht";
33     // Write the message into the write channel of the anonymous pipe
34     write(testpipe[1], &message, sizeof(message));
35 }
36
37 // Child process
38 if (pid_of_child == 0) {
39     printf("Child process: PID: %i\n", getpid());
40     // Block the write channel of the anonymous pipe testpipe
41     close(testpipe[1]);
42     // Create a receive buffer (80 bytes capacity)
43     char puffer[80];
44     // Read the message from the read channel of the anonymous pipe
45     read(testpipe[0], puffer, sizeof(puffer));
46     printf("Received: %s\n", puffer);
47 }
48 }
```

```
$ gcc anonymous_pipe_example.c -o anonymous_pipe_example
$ ./anonymous_pipe_example
Created the anonymous pipe testpipe.
Parent process: PID: 394769
Child process: PID: 394770
Received: Testnachricht
```

Named Pipes

- Processes, which are not closely related with each other, can communicate via **named pipes**
 - These pipes can be accessed by using their names
 - They are created in C by: `mkfifo("<pathname>", <permissions>)`
 - Any process, which knows the name of a pipe, can use the name to access the pipe and communicate with other processes
- The operating system ensures **mutual exclusion**
 - At any time, only a single process can access a pipe
- Named pipes are not erased automatically by the operating system (unlike anonymous pipes)

Size of named pipes in Linux

- Maximum number of bytes that can be written atomically (without interleaving with other writers) specifies the system variable `PIPE_BUF` and is typically equal to the page size $\implies$ `getconf PIPE_BUF /path`
Typically in Linux: 4096 bytes in a system with a page size of 4096 bytes
- Size (in bytes) is specified by the Kernel
Typically in Linux: 65536 bytes in a system with a page size of 4096 bytes
- The size cannot be modified dynamically. `F_SETPIPE_SZ` cannot be used with named pipes. The named pipe size is specified by the Kernel and not by the file `/proc/sys/fs/pipe-max-size`

Named Pipe Example (in C) – Part 1/4

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4 #include <fcntl.h>
5 #include <sys/stat.h>
6
7 void main() {
8     int pid_of_child;
9
10    // Create named pipe
11    if (mkfifo("testfifo", 0666) < 0) {
12        printf("Unable to create the named pipe.\n");
13        exit(1);
14    } else {
15        printf("Created the named pipe testfifo.\n");
16    }
17
18    // Create a child process
19    pid_of_child = fork();
20
21    if (pid_of_child < 0) {
22        perror("Unable to create the child process!\n");
23        exit(1);
24    }
```

The function call creates a file system entry named testfifo in the current directory. The first letter in the output of the ls command indicates that testfifo is a named pipe. The permissions are rw-r--r-- because umask is 022.

```
$ ls -la testfifo
```

```
prw-r--r--  1 bnc bnc    0  1. Feb 10:15 testfifo
```

Named Pipe Example (in C) – Part 2/4

```
25 // Parent process
26 if (pid_of_child > 0) {
27     printf("Parent process: PID: %i\n", getpid());
28
29     // Create the file descriptor (handle) for the pipe
30     int fd;
31
32     // Specify the message to be transferred
33     char message[] = "Testnachricht";
34
35     // Open the named pipe for writing
36     fd = open("testfifo", O_WRONLY);
37
38     // Write the message into the pipe
39     write(fd, &message, sizeof(message));
40
41     // Close the named pipe
42     close(fd);
43 }
```

Named Pipe Example (in C) – Part 3/4

```
44 // Child process
45 if (pid_of_child == 0) {
46     printf("Child process: PID: %i\n", getpid());
47
48     // Create the file descriptor (handle) for the pipe
49     int fd;
50     // Create a receive buffer
51     char puffer[80];
52
53     // Open the named pipe for reading
54     fd = open("testfifo", O_RDONLY);
55
56     // Read the message from the pipe
57     read(fd, puffer, sizeof(puffer));
58     printf("Received: %s\n", puffer);
59
60     // Close the named pipe
61     close(fd);
62
63     // Erase the named pipe
64     if (unlink("testfifo") < 0) {
65         printf("Unable to erase the named pipe.\n");
66         exit(1);
67     } else {
68         printf("The named pipe has been erased.\n");
69     }
70 }
71 }
```

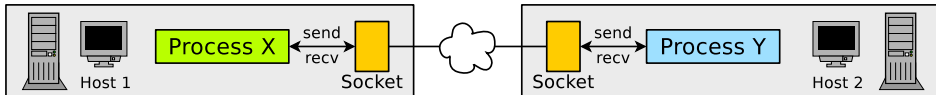
Named Pipe Example (in C) – Part 4/4

```
$ gcc named_pipe_example.c -o named_pipe_example
$ ./named_pipe_example
Created the named pipe testfifo.
Parent process: PID: 395415
Child process: PID: 395416
Received: Testnachricht
The named pipe has been erased.
```

You can monitor the named pipe in Linux/UNIX via `ls -l | grep <PID>` and inside the directory `/proc/<PID>/fd`

Sockets

- Full duplex-ready alternative to pipes and shared memory
 - Allow interprocess communication in distributed systems
- A user process can request a socket from the operating system and afterwards send and receive data via the socket
 - The operating system maintains all used sockets and the related connection information



- Ports are used for the communication via sockets
 - Port numbers are randomly assigned during connection establishment
 - Port numbers are assigned randomly by the operating system
 - Exceptions are port numbers of well-known applications, such as HTTP (80) SMTP (25), Telnet (23), SSH (22), FTP (21),...
- Sockets can be used in a blocking (synchronous) and non-blocking (asynchronous) way

Different Types of Sockets

- **Connectionless sockets (= datagram sockets)**
 - Use the Transport Layer protocol UDP
 - Advantage: Better data rate as with TCP
 - Reason: Lesser overhead for the protocol
 - Drawback: Segments may arrive in wrong sequence or may get lost
- **Connection-oriented sockets (= stream sockets)**
 - Use the Transport Layer protocol TCP
 - Advantage: Better reliability
 - Segments cannot get lost
 - Segments always arrive in the correct sequence
 - Drawback: Lower data rate as with UDP
 - Reason: More overhead for the protocol

Using Sockets

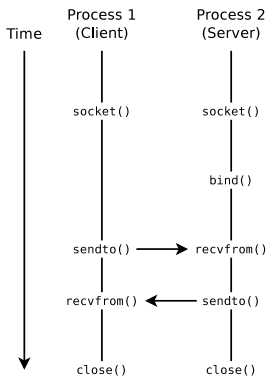
- Almost all major operating systems support sockets
 - Advantage: Better portability of applications
- Functions for communication via sockets:
 - Creating a socket:
`socket()`
 - Binding a socket to a port number and making it ready to receive data:
`bind()`, `listen()`, `accept()` and `connect()`
 - Sending/receiving messages via the socket:
`send()`, `sendto()`, `recv()` and `recvfrom()`
 - Closing a socket (releasing the file descriptor):
`close()`

In addition to the functions mentioned above, `shutdown()` can be used to terminate one or both communication directions. `SHUT_RD` = stop receiving data, `SHUT_WR` = stop sending data, `SHUT_RDWR` = disable both directions. Unlike `close()`, however, it does not release the file descriptor.

Overview of the sockets in Linux/UNIX: `netstat -n` or `lsof | grep socket`

Examples of interprocess communication via sockets (TCP and UDP) in Linux can be found on the website of this course

Connectionless Communication via Sockets – UDP



• Client

- Create socket (`socket`)
- Send (`sendto`) and receive data (`recvfrom`)
- Close socket (`close`)

• Server

- Create socket (`socket`)
- Bind socket to a port (`bind`)
- Send (`sendto`) and receive data (`recvfrom`)
- Close socket (`close`)

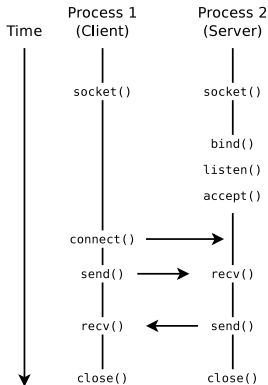
Connection-oriented Communication via Sockets – TCP

• Client

- Create socket (`socket`)
- Connect client with server socket (`connect`)
- Send (`send`) and receive data (`recv`)
- Close socket (`close`)

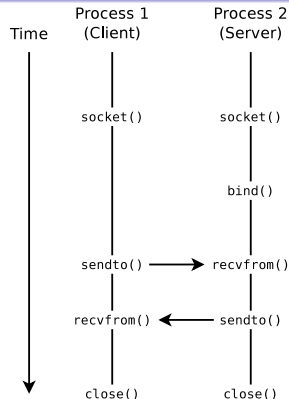
• Server

- Create socket (`socket`)
- Bind socket to a port (`bind`)
- Make socket ready to receive (`listen`)
 - Set up a queue for connection requests.
Specifies the number of connection requests, which can be stored in the queue
- Server accepts connections (`accept`)
 - Fetch the first connection request from the queue
- Send (`send`) and receive data (`recv`)
- Close socket (`close`)



Sockets via UDP – Example (Server)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <sys/socket.h>
5 #include <netinet/in.h>
6 #include <unistd.h>
7 #include <arpa/inet.h>
8
9 int main(int argc, char *argv[]) {
10     int sd, adresse_laenge;
11     char puffer[1024] = { 0 };
12     struct sockaddr_in adresse, client_adresse;
13     memset(&adresse, 0, sizeof(adresse));
14     memset(&client_adresse, 0, sizeof(client_adresse));
15     adresse.sin_family = AF_INET;
16     adresse.sin_addr.s_addr = INADDR_ANY;
17     adresse.sin_port = htons(atoi(argv[1]));
18
19     sd = socket(AF_INET, SOCK_DGRAM, 0);
20     bind(sd, (struct sockaddr *) &adresse, sizeof(adresse));
21     adresse_laenge = sizeof(client_adresse);
22     recvfrom(sd, (char *)puffer, sizeof(puffer), 0,
23             (struct sockaddr *) &client_adresse, &adresse_laenge);
24     printf("Empfangene Nachricht: %s\n", puffer);
25     char antwort[]="Server: Nachricht empfangen.\n";
26     sendto(sd, (const char *)antwort, sizeof(antwort), 0,
27           (struct sockaddr *) &client_adresse, adresse_laenge);
28     close(sd);
29     exit(0);
30 }
```



```
$ gcc udp_server.c -o udp_server
$ ./udp_server 50002
```

55/76

```
$ ./udp_server 50002
Empfangene Nachricht: Test
```

56/76

```
sequenceDiagram
    participant P1 as Process 1 (Client)
    participant P2 as Process 2 (Server)
    Note over P1: Time
    P1->>P1: socket()
    P1->>P2: connect()
    P1->>P2: send()
    P2->>P2: socket()
    P2->>P2: bind()
    P2->>P2: listen()
    P2->>P2: accept()
    P2->>P1: recv()
    P1->>P1: recv()
    P2->>P1: send()
    P1->>P1: close()
    P2->>P2: close()
```

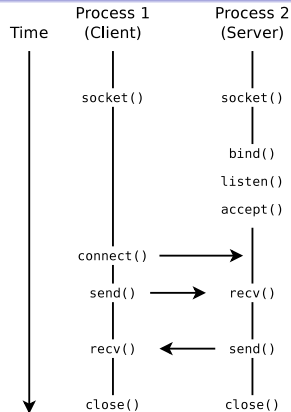
\$ gcc tcp_server.c -o tcp_server
\$./tcp_server 50003

Sockets via TCP – Example (Client)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <sys/socket.h>
5 #include <netinet/in.h>
6 #include <unistd.h>
7 #include <arpa/inet.h>
8
9 int main(int argc, char *argv[]) {
10     int sd;
11     char puffer[1024] = { 0 };
12     struct sockaddr_in adresse;
13     memset(&adresse, 0, sizeof(adresse));
14     adresse.sin_family = AF_INET;
15     adresse.sin_port = htons(atoi(argv[2]));
16     adresse.sin_addr.s_addr = inet_addr(argv[1]);
17
18     sd = socket(AF_INET, SOCK_STREAM, 0);
19     connect(sd, (struct sockaddr *) &adresse, sizeof(adresse));
20
21     printf("Bitte Nachricht eingeben: ");
22     fgets(puffer, sizeof(puffer), stdin);
23     write(sd, puffer, strlen(puffer));
24     memset(puffer, 0, sizeof(puffer));
25     read(sd, puffer, sizeof(puffer));
26     printf("%s\n", puffer);
27
28     close(sd);
29     exit(0);
30 }

```



```
$ gcc tcp_client.c -o tcp_client
$ ./tcp_client 127.0.0.1 50003
Bitte Nachricht eingeben: Test
Server: Nachricht empfangen.
```

```
$ ./tcp_server 50003
Empfangene Nachricht: Test
```


Semaphore

- In order to protect (lock) critical sections, not only the already discussed locks can be used, but also **semaphores**
- 1965: Published by Edsger W. Dijkstra
- A semaphore is a counter lock **S** with operations **P(S)** and **V(S)**
 - **V** comes from the Dutch *verhogen* = raise
 - **P** comes from the Dutch *proberen* = try (to reduce)
- The **access operations are atomic** $\implies$ cannot be interrupted (indivisible)
- May allow multiple processes accessing the critical section
 - In contrast to semaphores, locks ($\implies$ slide 14) can only be used to allow a single process to enter the critical section at the same time

Cooperating sequential processes. *Edsger W. Dijkstra* (1965)

<https://www.cs.utexas.edu/~EWD/ewd01xx/EWD123.PDF>

A Semaphore consists of 2 Data Structures

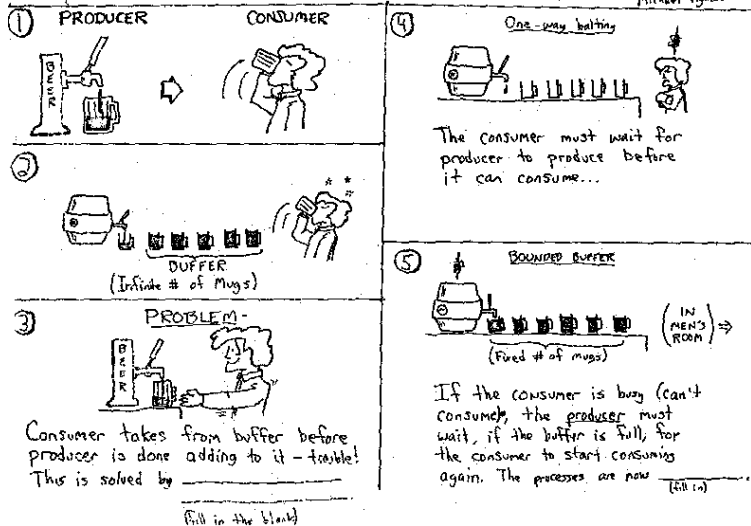
- ```
1 // apply the INIT operation on semaphore SEM
2 SEM.INIT(unsigned int init_value) {
3
4 // initialize the variable COUNT of Semaphor SEM
5 // with a non-negative initial value
6 SEM.COUNT = init_value;
7 }
```







Michael Vignaro



Source: Kenneth Baclawski (Northeastern University in Boston), Image source: Michael Vigneau (license: unknown)  
<http://www.ccs.neu.edu/home/kenb/tutorial/example.gif>

## Producer/Consumer Example (2/3)

- 3 semaphores are used to synchronize access to the buffer
  - empty
  - filled
  - mutex
- The semaphores filled and empty are used in opposite to each other
  - empty counts the number of empty locations in the buffer and its value is reduced by the producer (P operation) and raised by the consumer (V operation)
    - $\text{empty} = 0 \implies \text{buffer is completely filled} \implies \text{producer is blocked}$
  - filled counts the number of data packets (occupied locations) in the buffer and its value is raised by the producer (V operation) and reduced by the consumer (P operation)
    - $\text{filled} = 0 \implies \text{buffer is empty} \implies \text{consumer is blocked}$
- The semaphore mutex is used to ensure for the mutual exclusion

## Binary Semaphores

- **Binary semaphores** are initialized with value 1 and ensure that 2 or more processes cannot simultaneously enter their critical sections
- Example: The semaphore `mutex` from the producer/consumer example



Image Source: Carsten Vogt

- 
- Diagram illustrating the structure of a semaphore table:
- The table is indexed by **Group number** (0, 1, 2, 3, ..., n).
  - Each row represents a group and contains slots for semaphore numbers.
  - Row 0 contains slots 0 to 5, labeled  $S_{00}$  through  $S_{05}$ .
  - Row 1 contains slots 0 to 1, labeled  $S_{10}$  and  $S_{11}$ .
  - Row 2 contains slots 0 to 2, labeled  $S_{20}$ ,  $S_{21}$ , and  $S_{22}$ .
  - Row 3 contains slots 0 to 4, labeled  $S_{30}$ ,  $S_{31}$ ,  $S_{32}$ ,  $S_{33}$ , and  $S_{34}$ .
  - A bracket indicates that slots  $S_{30}$  through  $S_{34}$  form a **Semaphore group**.
  - An arrow points to  $S_{22}$ , labeled **individual semaphore**.
  - Row n is labeled **empty**.

- `semget()`: Create new semaphore or a group of semaphores or open an existing semaphore
- `semctl()`: Request or modify the value of an existing semaphore or of a semaphore group or erase a semaphore
- `semop()`: Carry out P and V operations on semaphores
- Information about existing semaphores (**System V**) provides the command `ipcs`



```

25 // Neue Semaphoregruppe 54321 mit einer Semaphore erstellen
26 returncode_semget2 = semget(sem_key2, 1, IPC_CREAT | IPC_EXCL | 0600);
27 if (returncode_semget2 < 0) {
28 printf("Die Semaphoregruppe %i konnte nicht erstellt werden.\n", sem_key2);
29 perror("semget");
30 exit(1);
31 }
32
33 // P-Operation definieren. Wert der Semaphore um eins dekrementieren
34 struct sembuf p_operation = {0, -1, 0};
35
36 // V-Operation definieren. Wert der Semaphore um eins inkrementieren
37 struct sembuf v_operation = {0, 1, 0};
38
39 // Erste Semaphore der Semaphoregruppe 12345 initial auf Wert 1 setzen
40 returncode_semctl = semctl(returncode_semget1, 0, SETVAL, 1);
41
42 // Erste Semaphore der Semaphoregruppe 54321 initial auf Wert 0 setzen
43 returncode_semctl = semctl(returncode_semget2, 0, SETVAL, 0);
44
45 // Initialen Wert der ersten Semaphore der Semaphoregruppe 12345 zur Kontrolle ausgeben
46 output = semctl(returncode_semget1, 0, GETVAL, 0);
47 printf("Wert der Semaphore mit ID %i und Key %i: %i\n", returncode_semget1, sem_key1, output);
48
49 // Initialen Wert der ersten Semaphore der Semaphoregruppe 54321 zur Kontrolle ausgeben
50 output = semctl(returncode_semget2, 0, GETVAL, 0);
51 printf("Wert der Semaphore mit ID %i und Key %i: %i\n", returncode_semget2, sem_key2, output);

```

Helpful documentation of `semctl`

<https://www.nt.th-koeln.de/fachgebiete/inf/diplom/semwork/unix/semctl/semctl.html>

```

52 // Einen Kindprozess erzeugen
53 pid_des_kindess = fork();
54
55 // Kindprozess
56 if (pid_des_kindess == 0) {
57 for (int i=0;i<5;i++) {
58 semop(returncode_semget2, &p_operation, 1); // P-Operation Semaphore 54321
59 // Kritischer Abschnitt (Anfang)
60 printf("B");
61 sleep(1);
62 // Kritischer Abschnitt (Ende)
63 semop(returncode_semget1, &v_operation, 1); // V-Operation Semaphore 12345
64 }
65 exit(0);
66 }
67
68 // Elternprozess
69 if (pid_des_kindess > 0) {
70 for (int i=0;i<5;i++) {
71 semop(returncode_semget1, &p_operation, 1); // P-Operation Semaphore 12345
72 // Kritischer Abschnitt (Anfang)
73 printf("A");
74 sleep(1);
75 // Kritischer Abschnitt (Ende)
76 semop(returncode_semget2, &v_operation, 1); // V-Operation Semaphore 54321
77 }
78 }

```

Helpful documentation of semop

<https://www.nt.th-koeln.de/fachgebiete/inf/diplom/semwork/unix/semop/semop.html>



```
$ gcc semaphore_beispiel_systemv.c -o semaphore_beispiel_systemv
$./semaphore_beispiel_systemv
Wert der Semaphore mit ID 98362 und Key 12345: 1
Wert der Semaphore mit ID 98363 und Key 54321: 0
ABABABABAB
Die Semaphorgruppe mit ID 98362 und Key 12345 wurde entfernt.
Die Semaphorgruppe mit ID 98363 und Key 54321 wurde entfernt.
```

```
$ printf "%d\n" 0x00003039 # Convert from hexadecimal to decimal
12345
$ printf "%d\n" 0x0000d431
54321
```

- 73/76

# Semaphores in Linux (System V vs. POSIX)

- The concept of protecting critical sections described so far is also called **System V semaphores** in the literature
- Some developers prefer the System V API and others the POSIX API... 🤔

C function calls of the POSIX semaphores specified in the header file `semaphore.h`

- `sem_init()`: Create a new **unnamed** semaphore and thereby specify the initial value
- `sem_open()`: Create a new **named** semaphore and thereby specify the initial value
- `sem_post()`: Increment the value of a semaphore (V operation)
- `sem_wait()`: Decrement the value of a semaphore (P operation). Blocking operation
- `sem_trywait()`: Decrement the value of a semaphore (P operation). Non-blocking operation
- `sem_timedwait()`: Decrement the value of a semaphore (P operation). Blocking operation but with a timeout
- `sem_getvalue()`: Request the value of a semaphore
- `sem_destroy()`: Erase an **unnamed** semaphore
- `sem_close()`: Close a **named** semaphore
- `sem_unlink()`: Erase a **named** semaphore
- Named POSIX semaphores are created in Linux in the folder `/dev/shm` with names of the form `sem.<name>`

One example of working of working with named POSIX semaphores in Linux can be found on the website of this course

# Mutexes

- If the semaphore feature of counting is not required, a simplified alternative, the mutex can be used instead
  - **Mutexes** (derived from **Mutual Exclusion**) are used to protect critical sections, which are allowed to be accessed by only **a single process** at any given moment
    - Mutexes can only have 2 states: **occupied** and **not occupied**
    - Mutexes have the same functionality as **binary semaphores**

Several implementations of the mutex concept exist

- **C standard library:** `mtx_init`, `mtx_unlock` ("V operation"), `mtx_lock` ("P operation"), `mtx_trylock`, `mtx_timedlock`, `mtx_destroy`
  - **POSIX threads:** `pthread_mutex_init`, `pthread_mutex_unlock`, `pthread_mutex_lock`, `pthread_mutex_trylock`, `pthread_mutex_timedlock`, `pthread_mutex_destroy`
  - **C standard library (Sun/Oracle Solaris):** `mutex_init`, `mutex_unlock`, `mutex_lock`, `mutex_trylock`, `mutex_destroy`
- 
- **Focus: Cooperation of threads of a process (intra-process synchronization)**
    - Cooperation of processes (inter-process synchronization) is not always possible and if so, then via a shared memory segment (System V or POSIX)

## Monitor and erase IPC Objects

- Information about existing (**System V**) shared memory segments, (**System V**) message queues and (**System V**) semaphores is provided by the command `ipcs`
- The easiest way to erase such shared memory segments, message queues and semaphores from the command line is the command `ipcrm`

```
ipcrm [-m shmids] [-q msgids] [-s semids]
 [-M shmkeys] [-Q msgkeys] [-S semkeys]
```

- **POSIX** memory segments and **POSIX** semaphores can be inspected and manually erased in the directory `/dev/shm`
- **POSIX** message queues can be inspected and manually erased in the directory `/dev/mqueue`